DAEDALUS

Procedural Maze Engine

PLUGIN ARCHITECTURE SPECIFICATION

Version 1.0 — May 2026

Abstract. The Daedalus plugin system delivers a production-grade, Spring-native extension framework for the maze generation and solving platform. Plugins can register new generators and solvers, subscribe to lifecycle and gameplay events, expose REST endpoints and UI themes, and participate in the full Spring application context — all discovered via standard Java ServiceLoader with optional external JAR loading and explicit lifecycle management.

Document Status: Final — Approved for Implementation

Table of Contents

1. Introduction & Design Goals

Daedalus is a modular, Spring Boot-based engine for procedural generation of mazes, dungeons, and related content. The plugin architecture was designed from the ground up to satisfy four primary goals:

• **Extensibility without Forking** — Third parties must be able to add new generators, solvers, visual themes, and even gameplay mechanics without modifying the core engine.

• **Spring-Native Integration** — Plugins are not isolated black boxes; they receive full access to the Spring ApplicationContext, event bus, and can contribute their own @Beans, @RestControllers, and @EventListeners.

• **Explicit & Observable Lifecycle** — Every plugin goes through a well-defined state machine (DISCOVERED → INITIALIZED → REGISTERED → STARTED → STOPPED) with clear hooks and failure semantics.

• **Production Safety** — Strong typing via registries, dependency declaration, resource cleanup contracts, and the ability to run external plugins under a separate classloader.

Design Note: The architecture deliberately avoids heavy OSGi or custom module systems. Instead it leverages the battle-tested Java ServiceLoader for discovery and Spring's own extension points for runtime wiring. This keeps the cognitive load low while still delivering enterprise-grade capabilities.

2. Core Components

2.1 MazePlugin — The Service Provider Interface

MazePlugin is the sole extension point. All plugins must implement this interface (directly or via AbstractPlugin). It uses default methods so implementors only override what they need.

```
package com.daedalus.plugin;

import com.daedalus.model.AlgorithmDescriptor;
import java.util.List;

public interface MazePlugin {

PluginManifest manifest();

default void init(PluginContext ctx) {}

default void registerAlgorithms(PluginContext ctx) {}

default void start(PluginContext ctx) {}

default void stop(PluginContext ctx) {}

default List<AlgorithmDescriptor> contributedAlgorithms() { return List.of(); }
}
```

Lifecycle contract: init → registerAlgorithms → start → … → stop. The framework guarantees this ordering and that stop is called even on failure paths (via try/finally).

2.2 PluginManifest — Metadata & Dependency Declaration

```
public record PluginManifest(
String id, // unique slug, e.g. "biome-generators"
String displayName,
String version,
String author,
String description,
String[] requires // ids of other plugins loaded first
) {
public PluginManifest(...) { this(..., new String[0]); }
}
```

Manifests are exposed via GET /api/plugins and used by the PluginManager to enforce load order and detect missing dependencies before any plugin code executes.

2.3 PluginContext — The Service Handle

A lightweight, final handle passed to every lifecycle method. It deliberately exposes only four capabilities, keeping the plugin API narrow and auditable:

```
public final class PluginContext {
private final ApplicationContext spring;
private final GeneratorRegistry generators;
private final SolverRegistry solvers;

public GeneratorRegistry generators() { return generators; }
public SolverRegistry solvers() { return solvers; }
public ApplicationContext beans() { return spring; }
public ApplicationEventPublisher events() { return spring; }

public <T> T bean(Class<T> type) { return spring.getBean(type); }
}
```

Why not inject everything? Explicit methods make it obvious what a plugin is allowed to do and simplify testing/mocking of the context itself.

2.4 AbstractPlugin & 2.5 PluginLifecycle

```
public abstract class AbstractPlugin implements MazePlugin {
protected PluginContext context;

@Override
public void init(PluginContext ctx) {
this.context = ctx;
}
}

public enum PluginLifecycle {
DISCOVERED, INITIALIZED, REGISTERED, STARTED, STOPPED, FAILED
}
```

The enum is used internally by the PluginManager to track progress and to decide whether a failed plugin should block startup or be isolated.

3. Plugin Lifecycle & State Transitions

The framework maintains a strict state machine for every discovered plugin:

State	Trigger	Hook Called	Guarantees
DISCOVERED	ServiceLoader + manifest parse	—	Manifest validated, dependencies checked
INITIALIZED	All dependencies STARTED	init(ctx)	context != null, safe to store
REGISTERED	After init batch	registerAlgorithms(ctx)	Generators/solvers visible to engine
STARTED	Spring context refreshed	start(ctx)	Full event bus & REST active
STOPPED	Shutdown hook or explicit	stop(ctx)	Resources released, listeners unsubscribed
FAILED	Any exception in prior phase	—	Isolated; other plugins continue

Failure Semantics: A plugin that throws in registerAlgorithms is marked FAILED and its contributions are rolled back. The rest of the system continues. This allows graceful degradation (e.g., a optional visual theme plugin can fail without breaking core maze generation).

4. Extension Points in Detail

4.1 Algorithm Registration

The primary purpose of most plugins is to contribute new Generator or Solver implementations. The PluginContext provides typed registries:

```
// Inside registerAlgorithms(PluginContext ctx)
ctx.generators().register(new FractalCaveGenerator());
ctx.solvers().register(new AStarWithBiomeHeuristic());
```

4.2 Event Subscription

Because events() returns the Spring ApplicationEventPublisher, plugins can both publish custom events and subscribe via @EventListener methods on their own Spring beans (or even on the plugin class itself if it is a @Component).

4.3 Full Spring Participation

Plugin JARs are added to the classloader *before* the Spring context is refreshed. Therefore a plugin can contain any number of @Configuration, @RestController, @Service, or @EventListener classes that will be picked up automatically. The only requirement is that the plugin's package is scanned (usually by listing it in the core @SpringBootApplication scanBasePackages or by using a marker annotation).

4.4 UI / AlgorithmDescriptor Contribution

The optional contributedAlgorithms() method returns metadata used by the web UI to list available algorithms without instantiating them. This keeps the plugin lightweight until the user actually selects it.

5. Discovery, ClassLoading & Deployment Model

Built-in Plugins

Core plugins are declared in META-INF/services/com.daedalus.plugin.MazePlugin inside the main application JAR. They are loaded with the application classloader.

External Plugins

Drop a JAR into the plugins/ directory next to the application JAR. At startup the PluginManager:

• Scans plugins/*.jar for a valid manifest and ServiceLoader entry.

• Creates a new URLClassLoader for the plugin (parent = application loader) — strong isolation of plugin-private classes.

• Registers the plugin’s SPI implementation.

• Proceeds with normal lifecycle.

Hot-Reload Note: Current design does not support runtime reloading of plugin JARs. A restart is required. Future versions may add a watched directory + dynamic classloader swap with quiescence guarantees.

6. Worked Example: FractalBiomePlugin

Below is a minimal but complete plugin that adds three new biome-aware generators and listens for generation events to log statistics.

```
package com.example.fractalbiome;

import com.daedalus.plugin.*;
import org.springframework.context.event.EventListener;
import org.springframework.stereotype.Component;

@PluginManifest(
id = "fractal-biome",
displayName = "Fractal Biome Generators",
version = "1.2.0",
author = "Daedalus Labs",
description = "Adds cave, forest and crystal biome generators"
)
public class FractalBiomePlugin extends AbstractPlugin {

@Override
public void registerAlgorithms(PluginContext ctx) {
ctx.generators().register(new FractalCaveGenerator());
ctx.generators().register(new ForestCanopyGenerator());
ctx.generators().register(new CrystalCavernGenerator());
}

@Component
public static class GenerationStatsListener {
@EventListener
public void onMazeGenerated(MazeGeneratedEvent e) {
// custom telemetry, metrics, etc.
System.out.printf("Generated %s in %dms%n", e.getAlgorithm(), e.getDuration());
}
}
}
```

The @PluginManifest annotation (or a plugin.properties file) is read by the discovery layer to populate the PluginManifest record without forcing the plugin class to be instantiated early.

7. Best Practices, Security & Isolation

• **Resource Management** — Always release threads, file handles, and Spring subscriptions in stop(). The framework does not guarantee GC will run promptly after plugin unload.

• **Minimal Surface** — Prefer registering algorithms over exposing full controllers. Every additional Spring bean increases the attack surface.

• **Versioning** — Follow semantic versioning in the manifest. The requires array should pin major versions only.

• **Exception Hygiene** — Never let checked exceptions escape lifecycle methods. Wrap and log; let the framework mark the plugin FAILED.

• **Classloader Hygiene** — Do not store static references to classes loaded from the plugin classloader; they become unreachable after stop and can cause memory leaks.

• **Testing** — Unit test your plugin logic in isolation. Integration tests should use a test PluginContext that supplies mock registries and an in-memory Spring context.

8. Formal Guarantees & Audit Surface

The plugin system provides the following formal properties that can be audited or proven:

• **Ordering Invariant** — For any two plugins A and B, if A appears in B.requires, then A reaches STARTED before B.init is called (proven by topological sort in PluginManager).

• **Isolation** — Classes defined inside an external plugin JAR are never visible to other plugins or the core unless explicitly exported via the plugin's package list.

• **Rollback** — Any exception thrown from registerAlgorithms triggers automatic un-registration of any partial contributions made by that plugin before the exception.

• **Event Delivery** — All @EventListener methods contributed by a plugin receive events only after the plugin has reached STARTED state.

Audit Surface: The only runtime reflection used is ServiceLoader and Spring's standard component scanning. No custom bytecode manipulation or unsafe operations are performed on plugin code.

9. Appendix: Full Source Listings

The following listings are the exact source files distributed with Daedalus 1.0.

PluginContext.java

```
package com.daedalus.plugin;

import com.daedalus.engine.generators.GeneratorRegistry;
import com.daedalus.solver.solvers.SolverRegistry;
import org.springframework.context.ApplicationContext;
import org.springframework.context.ApplicationEventPublisher;

public final class PluginContext {
private final ApplicationContext spring;
private final GeneratorRegistry generators;
private final SolverRegistry solvers;

public PluginContext(ApplicationContext spring,
GeneratorRegistry generators,
SolverRegistry solvers) {
this.spring = spring;
this.generators = generators;
this.solvers = solvers;
}

public GeneratorRegistry generators() { return generators; }
public SolverRegistry solvers() { return solvers; }
public ApplicationContext beans() { return spring; }
public ApplicationEventPublisher events() { return spring; }

public <T> T bean(Class<T> type) {
return spring.getBean(type);
}
}
```

MazePlugin.java (abridged)

```
package com.daedalus.plugin;

import com.daedalus.model.AlgorithmDescriptor;
import java.util.List;

public interface MazePlugin {
PluginManifest manifest();
default void init(PluginContext ctx) {}
default void registerAlgorithms(PluginContext ctx) {}
default void start(PluginContext ctx) {}
default void stop(PluginContext ctx) {}
default List<AlgorithmDescriptor> contributedAlgorithms() { return List.of(); }
}
```

End of Specification

Questions? Contact plugins@daedalus.engine